RPG IV Mise en œuvre

Types de données

Généralités

- RPG IV permet de manipuler des données typées
- Chaque type possède
  - Son domaine de valeur
  - Des initialisations propres
  - Une représentation interne propre

Types de données

Type	Description
A	Caractère
N	Indicateur, booléen. Codé comme un caractère
G	Caractère graphic
C	Caractère UCS-2
B	Numérique binaire
F	Nombre réel (notation scientifique)
I	Entier binaire signé
P	Numérique condensé
U	Entier binaire non signé
D	Date
T	Heure
Z	Horodatage
*	Pointeur

Domaines de valeur (1/2)

Type	Minimum	Maximum
A	1	16.773.104
N	1	16.773.104
G	1	8.386.552
C	1	8.386.552
9B	-999.999.999	+999.999.999
4F	-3.4028235E+38	+3.4028235E+38
8F	-1.797693134862315E+308	+1.797693134862315E+308
3I0	-127	+128
5I0	-32.768	+32.767
10I0	-2.147.483.648	+2.147.483.647
20I0	-9.223.372.036.854.775.808	+9.223.372.036.854.775.807

Domaines de valeur (2/2)

Type	Minimum	Maximum
63P0	-9999…999	+999…999
3U0	0	+255
5U0	0	+65.535
10U0	0	+4.294.967.295
20U0	0	+18.446.744.073.709.551.615
D	*YMD/*DMY/*MDY/*JUL : 1940-01-01 *CYMD/*CDMY/*CMDY : 1900-01-01 *ISO/*USA*/*EUR/*JIS/*LONGJUL : 0001-01-01	*YMD/*DMY/*MDY/*JUL : 2039-12-31 *CYMD/*CDMY/*CMDY : 2899-12-31 *ISO/*USA*/*EUR/*JIS/*LONGJUL : 9999-12-31
T	USA : 00:00 AM Autres : 00:00:00	USA : 12:00 AM Autres : 24:00:00
Z	0001-01-01-00.00.00.000000	9999-12-31-24.00.00.000000
*	*NULL	Adresse maximale du travail

Initialisations (1/3)

- Le mot-clé INZ permet d'indiquer une valeur initiale pour une variable
  - Sauf les paramètres reçus
- INZ prend en paramètre la valeur
  - Dans sa représentation alphanumérique
- La valeur doit être connue à la compilation
  - Il est possible d'utiliser des BIFs tant que le résultat est connu à la compilation

    %size

    %addr

    ...

Initialisations (2/3)

```
 * alphanumérique
D Alpha            S            25A   INZ( 'Une valeur alpha' )
 * alphanumérique varying
D AlphaVar2        S            25A   VARYING    INZ( 'Une valeur alpha' )
D AlphaVar4        S        100000A   VARYING(4) INZ( 'Une autre valeur' )

 * Indicateur
D EstEnErreur1     S             N    INZ( *ON )
D EstEnErreur3     S             N    INZ( '1' )

 * Numérique binaire
D Binaire          S            9B 0 INZ( 0 )

 * Nombre réel
D Reel             S            4F   INZ( 12.458 )
D Reel2            S            4F   INZ( 4.23E-5 )

 * Entier binaire signé
D EntierS          S            3I 0 INZ( -3 )

 * Entier binaire non signé
D EntierNS         S            5U 0 INZ( 12 )
```

VARYING(4) pour les valeurs de plus de 65.535 caractères

*ON / ‘1’
*OFF / ‘0’

Notation scientifique = 4,32*10^-5

Initialisations (3/3)

```
h datfmt( *iso ) timfmt( *iso )

* Numérique condensé
D Condense          S              2P 1 INZ( 1.2 )

 * Numérique étendu
D Etendu            S              4S 2 INZ( 12.34 )

 * Date
D DateSys           S               D   INZ( *sys )
D DateIso           S               D   INZ( d'2010-05-25' )
D DateEur           S               D   INZ( d'2010-05-25' )
D                                       DATFMT( *EUR)

 * Heure
D HeureSys          S               T   INZ( *sys )
D HeureIso          S               T   INZ( t'10.25.00')
D HeureUsa          S               T   INZ( t'10.25.00')
D                                       TIMFMT( *USA )

 * Horodatage
D HoroDate          S               Z   INZ( z'2010-05-25-14.08.25.123456' )

 * Pointeur
D Pointeur          S               *   INZ( *NULL )
D PointeurFou       S               *   INZ( %addr( Etendu ) )
```

*sys = date d'exécution
*job = date du travail

Le format de date est celui indiqué par DATFMT en carte H, même si la variable a un format différent

Idem, voir TIMFMT de la carte H

La valeur retournée par %addr est connue à la compilation car c'est une variable locale

Représentation interne

- Le format de stockage des valeurs manipulées par les variables
- Ce format est propre à chaque type de donnée
- Certains types s’appuient sur d’autres
  - Les dates, heures et horodatages sont stockés dans leur représentation alphanumérique

Alphanumérique (1/2)

- Type A
  - Chaque octet représente un caractère
  - La valeur dépend du CCSID
- Type A VARYING
  - VARYING ou VARYING(2)

    Les 2 premiers octets donnent la taille actuelle
  - VARYING ou VARYING(4)

    Les 4 premiers octets donnent la taille actuelle

Alphanumérique (2/2)

```
 * alphanumérique
D Alpha            S              25A   INZ( 'Une valeur alpha' )
 * alphanumérique varying
D AlphaVar2        S              25A   VARYING    INZ( 'Une valeur alpha' )
D AlphaVar4        S          100000A   VARYING(4) INZ( 'Une autre valeur' )
```

16 caractères

	Rang octet	1	2	3	4	5	6	7	8	…	16
Alpha	Hexa	E4	95	85	40	A5	81	93	85		81
	Alpha/Num	U	N	e		V	a	l	e		a
AlphaVar2	Hexa	00	10	E4	95	85	40	A5	81		97
	Alpha/Num	16		U	n	e		v	a		p
AlphaVar4	Hexa	00	00	00	10	E4	95	85	40		81
	Alpha/Num		16			U	n	e			a

Numérique (1/2)

- De nombreux types, que l'on peut classer en 2 catégories
  - Stockage binaire
    - Types I, U, B, F
  - Stockage alphanumérique
    - Types S, P
- Remarque
  - Les formats binaires sont plus performants en calcul
    - Pas de conversion à réaliser avant et après le calcul
  - Les formats alphanumériques sont les seuls à proposer une virgule fixe

Numérique (2/2)

Type	Taille en octets
9B	4
4F	4
8F	8
3I0	1
5I0	2
10I0	4
20I0	8
3U0	1
5U0	2
10U0	4
20U0	8
2S0	2
2P0	2
3P0	3

Numérique binaire (1/3)

- Types U et B
  - Entier non signé
  - A chaque bit correspond la valeur $2^{\text{rang du bit}}$
- Type I
  - Entier signé
  - Valeurs positives

    1er bit de gauche = 0

    A chaque bit correspond la valeur $2^{\text{rang du bit}}$
  - Valeurs négatives

    1er bit de gauche = 1

    La valeur de complémentation pour un octet à ’00’

Numérique binaire (2/3)

- Type F
  - Réel

    Stocké sous la forme mantisse + exposant

    Valeur = mantisse * $10^{exposant}$

    Mantisse et exposant stockés en format entier signé

Numérique binaire (3/3)

```
 * Même valeur pour tous les types pour simplifier la comparaison
D Binaire          S              3B 0 INZ( 12 )
D EntierNS         S              3U 0 INZ( 12 )
D EntierSP         S              3I 0 INZ( 12 )
D EntierSN         S              3I 0 INZ( -12 )
```

Une longueur de 3 correspond à 1 octet

	Rang bit	7	6	5	4	3	2	1	0	Total
Binaire	Valeur	0	0	0	0	1	1	0	0	
	2^{Rang}					8	4			12
EntierS	Valeur	0	0	0	0	1	1	0	0	
	2^{Rang}					8	4			12
EntierSP	Valeur	0	0	0	0	1	1	0	0	
	2^{Rang}					8	4			12
EntierSN	Valeur	1	1	1	1	0	1	0	0	
	Complément	0	0	0	0	1	1	0	0	
	2^{Rang}					8	4			-12

Numérique alphanumérique (1/2)

- Type S : étendu
  - Stocké sous forme de caractère
  - 1 octet = 1 caractère = 1 chiffre
  - Perte de place car

    1 octet = 256 valeurs possibles

    Pour stocker 10 valeurs possibles par chiffres
- Type P : condensé
  - 1 octet permet de stocker 2 chiffres

    Suppression des 4 bits fixes pour représenter les chiffres sous forme de caractères
  - 4 bits sont ajoutés pour stocker le signe

    2P0 1 octet + 4 bits => 2 octets

    3P0 1 octet + 4 bits + 4 bits => 2 octets

Numérique alphanumérique (2/2)

```
 * Numérique étendu
D Etendu          S              4S 0 INZ( 12 )
 * Numérique condensé
D Condense        S              4P 0 INZ( 12 )
```

	Rang octet	1	2	3	4
Etendu	Hexa	F0	F0	F1	F2
	Alpha/Num	0012			
Condensé	Hexa	00	01	2F	
	Alpha/Num	00012		+	